**MOBILE APPLICATION DEVELOPMENT**

**Exercise Sheet**

**Exercise: Develop a Mobile Application Using a REST API**

---

**Scenario**

A university wants to provide students with a simple mobile application that allows them to check the current weather conditions of different cities.

The university has decided to develop a **Weather Information Mobile Application** that retrieves weather information from an external **REST API**.

The application should allow a user to enter the name of a city and request its current weather information. The Android application must send a request to the weather API, receive the response in **JSON format**, process the required data, and display the information clearly to the user.

For example, a user may enter **Colombo** and request the current weather information. The application should retrieve the relevant information from the API and display the city, temperature, weather condition, humidity, and wind speed.

The application should also provide appropriate messages when the user enters an invalid city or when the application cannot communicate with the API.

---

**Student Task**

Develop an Android application named **Weather App** that communicates with a REST API and displays weather information retrieved from the API.

You are required to complete the following tasks.

---

**Task 01 – Design the User Interface**

Create a suitable user interface for the Weather App.

The interface should contain:

- Application title
- Text field to enter a city name
- **Search Weather** button

- Area to display the weather information

The application should display at least the following information:

- City Name
- Temperature
- Weather Condition
- Humidity
- Wind Speed

You may use appropriate Android UI components such as TextView, EditText, Button, ImageView, and suitable layouts.

---

**Task 02 – Select a REST API**

Select a suitable weather REST API that provides current weather information.

Find and identify:

- API name
- API endpoint
- HTTP method
- Required parameters
- API key requirement
- JSON response structure

**Important:** Read the API documentation before starting the Android implementation.

---

**Task 03 – Test the API**

Use **Postman** to test the selected API.

Perform a **GET** request using a valid city.

Verify that:

- The request is successfully sent.
- The server returns a successful response.

- The response is provided in JSON format.
- The required weather information is available in the response.

Take a screenshot of the Postman request and response.

---

**Task 04 – Create the Android Application**

Create the Weather App using **Android Studio and Kotlin**.

Configure the application so that it can communicate with an external web service.

---

**Task 05 – Send an API Request**

Implement the API request in your Android application.

When the user clicks the **Search Weather** button:

1. Read the city name entered by the user.
2. Validate the input.
3. Construct the API request.
4. Send a **GET** request to the weather API.
5. Receive the response from the server.

---

**Task 06 – Process the JSON Response**

Examine the JSON response received from the API.

Identify the fields containing:

- City name
- Temperature
- Weather condition
- Humidity
- Wind speed

Parse the JSON response using a suitable approach/library.

Extract the required values from the response.

---

**Task 07 – Display the API Data**

Display the information retrieved from the API in the Android application.

For example:

**Colombo**

**29°C**

Cloudy

Humidity: 78%

Wind Speed: 12 km/h

The values displayed in the application must come from the **API response**.

Do not use permanently hard-coded weather values.

---

**Task 08 – Handle Errors**

Implement appropriate error handling.

Your application should handle at least the following situations:

**Case 1 – Empty City Name**

If the user clicks the button without entering a city, display an appropriate message.

**Case 2 – Invalid City**

If the entered city does not exist, display an appropriate error message.

**Case 3 – Network Error**

If the device cannot connect to the API, display an appropriate message.

**Case 4 – API Error**

If the API returns an unsuccessful response, handle the error appropriately.

---

**Task 09 – Test the Application**

Test your application using at least **three different cities**.

For example:

- Colombo
- Kandy
- London

Also test:

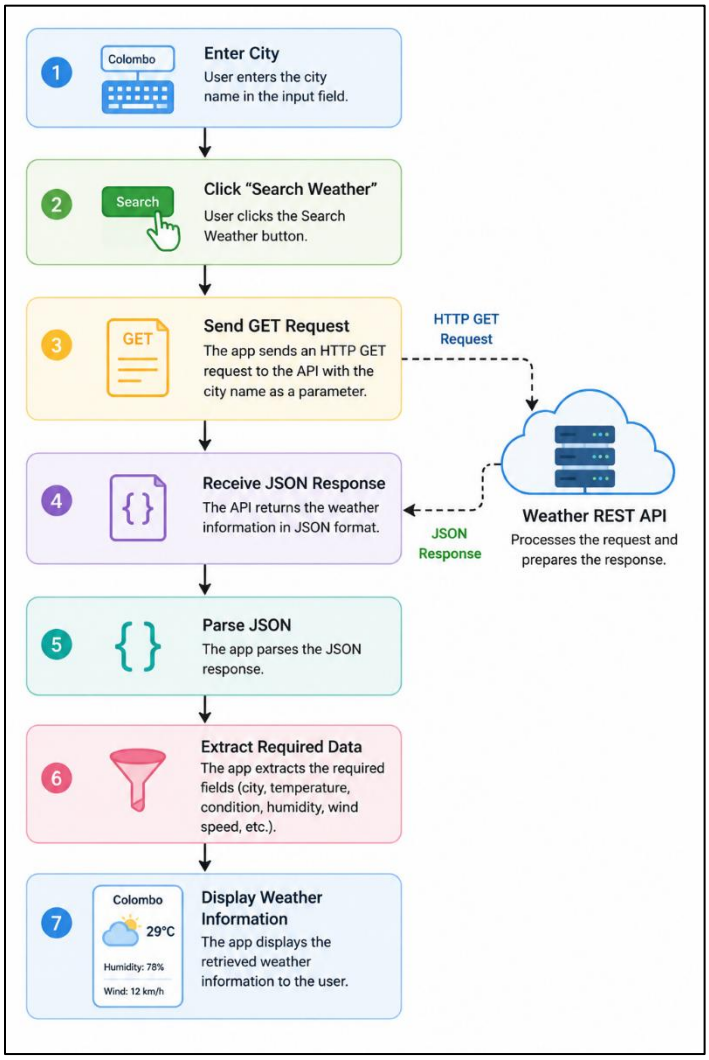
- An invalid city name
- Empty input
- Network/API failure if possible

Record whether the application produces the expected result.

---

**Expected Application Flow**

The application should follow the following process:



---

**Final Requirements**

Your completed application must:

- Use a **REST API**.
- Use the **GET** HTTP method.
- Receive data in **JSON format**.
- Parse the JSON response.
- Display API data dynamically.
- Allow the user to search for different cities.
- Include appropriate error handling.
- Have a clear and user-friendly interface.

---

**Submission**

Submit the following:

1. **Android Studio project**
2. Screenshot of the application interface
3. Screenshot of a successful weather result
4. Screenshot of the API request and JSON response in Postman
5. Screenshot of error handling
6. A short description of the API used